

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

This document provides an in-depth technical walkthrough of every layer of the Margdarshan.ai grievance redressal system, intended for technical evaluators, contributors, and future maintainers. It covers the problem context, end-to-end pipeline, machine learning models, software architecture, frontend, backend, database schema, evaluation, comparative analysis with existing systems, and the future roadmap.

Contents

1. Problem Statement and Solution Overview
2. System Architecture at a Glance
3. Citizen-Facing Flow - Layer by Layer
4. Department / Government Dashboard Flow - Layer by Layer
5. AI / ML / NLP Foundations Used in the System
6. Model Training, Evaluation, and Accuracy
7. Software Engineering Concepts in Play
8. Comparative Analysis with Existing Systems
9. Future Enhancements and Roadmap
10. Appendix - APIs, Schemas, Deployment

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

1. Problem Statement and Solution Overview

The problem. India runs more than seventy public grievance portals across central ministries, state departments, and city utilities. A typical citizen with a real complaint - say a power cut in Bhopal or a missing pension credit - has to (a) identify the correct portal out of dozens, (b) read English-only instructions, (c) fill long forms with portal-specific fields they may not understand, and (d) keep checking different websites for status. The result: most genuine grievances either never get filed or get filed at the wrong portal where they are eventually rejected without action. Duplicate filings and "complaint fatigue" further clog official systems and drown out signal.

Our solution. Margdarshan.ai (Hindi *margdarshan*: guidance / showing the way) is an AI-driven, multilingual conversational platform that lets any citizen describe a problem in Hindi, English, or Hinglish - in plain everyday language - and automatically performs the entire downstream pipeline: language understanding, department classification, location-aware portal routing, semantic deduplication, portal-specific field collection, submission, periodic status tracking, and resolution notification. The same backend powers a second product surface - a Department / Government Dashboard - where reviewers monitor complaints, audit AI decisions, override classifications, and watch tamper-evident event chains.

Concretely the platform does:

- Accepts complaints in Hindi (Devanagari), English, and Hinglish (Roman-script Hindi) - including voice input.
- Uses a large language model (Claude Haiku 4.5) to extract complaint structure and slot-fill missing details.
- Classifies into 25 departments and 142 sub-categories with a fine-tuned MuRIL transformer (with a rule-based fallback).
- Routes to the most specific portal among 70 registered government portals (regional > state > central hierarchy).
- Deduplicates against past complaints using S-BERT multilingual embeddings (cosine threshold 0.85).
- Collects portal-specific fields conversationally - rather than dropping the user into another website.
- Submits via portal-specific adapters (CPGRAMS, MP CM Helpline 181, MPPKVVCL) with 7-attempt exponential backoff.
- Polls each portal at adaptive intervals (1h to 24h) and notifies citizens in their own language over WhatsApp.
- Maintains a SHA-256 hash-chained audit trail so every decision is verifiable end-to-end.
- Surfaces a real-time admin dashboard with stats, audit chains, AI review controls, and routing explanations.

Why this matters:

Public grievance redressal is the most direct interface a citizen has with the state. Today that interface is fragmented, English-biased, and indifferent to context. Margdarshan.ai collapses the discovery and submission cost to a single conversation, while keeping every decision auditable so it can survive regulatory scrutiny.

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

2. System Architecture at a Glance

Margdarshan.ai is implemented as ten independent containerised services orchestrated by Docker Compose, with Redis Pub/Sub as the event bus and PostgreSQL as the source of truth. The system is built on the **microservices** pattern - each stage of the pipeline lives in its own service so it can be deployed, scaled, debugged, and replaced independently.

The ten services and their ports:

Service	Port	Role
postgres	5432	Source of truth for users, complaints, messages, audit chain, reviews.
redis	6379	Pub/Sub event bus, session cache, retry queues, feedback queue.
service-gateway	8000	Public API, JWT auth, pipeline orchestrator, audit chain writer.
service-chatbot	8001	Thin conversation interface. Proxies every turn to NLU.
service-location	8002	Reverse geocoding (LocationIQ + OSM Nominatim) and pincode lookup.
service-nlu	8003	Conversation brain: state machine, decision tree, Claude-Haiku NLU.
service-classifier	8004	Department/sub-category/priority/sentiment classifier (MuRIL or rule-based).
service-routing	8005	Portal lookup - 70 portals, hierarchical Regional/State/Central matching.
service-submission	8006	Portal adapters (CPGRAMS, MPCM181, MPPKVVCL, generic mock).
service-tracker	8007	Adaptive status polling + bilingual WhatsApp notifications.
web-frontend	5173	React + Vite SPA - citizen chat UI and admin dashboard.

The pipeline: citizen message -> chatbot -> NLU -> classifier -> routing -> dedup -> field collection -> submission -> tracker -> notification -> citizen.

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

Why microservices and not a monolith?

(a) Different stages have radically different runtime profiles - the classifier needs PyTorch and 1.5 GB of RAM for MuRIL inference, the chatbot is largely I/O-bound talking to Anthropic, and the location service is just an HTTP proxy. Putting them together would lock the whole system to the heaviest dependency set. (b) Independent deploy: when we re-train MuRIL we only rebuild service-classifier. (c) Independent scaling: the chatbot can be scaled horizontally without touching the database-bound gateway. (d) Independent failure: if the tracker poller is degraded, complaint submission still works. (e) Clear contracts: each service exposes a small typed FastAPI surface, which becomes the integration contract.

Why Redis Pub/Sub for the event bus?

Redis is already in the stack as a session and cache store, so adding Pub/Sub is zero extra ops. We don't need Kafka-grade durability for a hackathon prototype, and the events we care about (stage transitions of a single complaint over a few minutes) fit naturally in pub-sub. The orchestrator listens on a single pipeline:all channel and routes events to the appropriate next-stage trigger. If we ever need at-least-once delivery for audit-critical stages we can swap the bus for Kafka or Redis Streams without changing service code - the event schema is already a typed Pydantic model in `shared-schema/events.py`.

Why PostgreSQL?

We need (a) ACID guarantees for the audit chain - inserting an event and its hash must be transactional, (b) JSONB to store the entire UCO (Unified Complaint Object) pipeline state without rigid column schemas, (c) array columns to store 384-dimensional S-BERT embeddings for semantic dedup, and (d) battle-tested admin tooling. Postgres gives us all four. A document database (MongoDB) would have made the audit chain harder to reason about and lost ACID; a pure relational design would have made the rapidly-evolving UCO blob painful.

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

3. Citizen-Facing Flow - Layer by Layer

The citizen flow is split into seven logical layers running across two services on the backend, plus the React frontend. Every user message travels through all seven layers before producing a reply.

Layer 1 - Frontend Chat Interface (React)

Concept. A three-panel single-page application: left panel lists the citizen's past complaints, centre panel is the conversational chat (text + voice), right panel is a live pipeline visualisation that lights up each stage in real time as backend events arrive over WebSocket.

Stack and why we chose it:

- **React 18 + Vite** - fast HMR, modern JSX runtime, no Webpack baggage. Vite is dramatically faster than CRA for dev iteration.
- **TypeScript 5.6** - the pipeline payloads are non-trivial typed unions; TS catches integration drift between gateway and UI.
- **Tailwind CSS 3.4** - utility-first lets a small team converge on a consistent visual language without bikeshedding stylesheet structure.
- **React Context (no Redux)** - the global state we actually need (auth, chat, pipeline) is small and event-driven; Redux would be ceremony for no benefit.
- **react-i18next** - Hindi / English / Hinglish swapping with browser auto-detect and a clean t('key') call site.
- **Leaflet + OpenStreetMap** - the location pinning modal needs a free, offline-friendly map tile source; Leaflet is the de-facto standard with React Leaflet bindings.
- **Web Speech API** - native browser voice input (hi-IN, en-IN). No third-party STT cost, works fully on-device.

How the UI talks to the backend:

All HTTP calls go through `src/lib/api.ts` against the gateway URL from `VITE_GATEWAY_URL`. Auth state is a JWT in `localStorage` under key `margdarshan_token`. The live pipeline panel subscribes over `ws://gateway/ws/pipeline/{complaint_id}` and receives JSON `PipelineEvent` messages on every stage transition.

Why not Next.js / SSR? The app is authenticated and dynamic by definition - there is no SEO surface, no static content to render. SPA + REST + WebSocket is the simplest correct choice.

Layer 2 - Chatbot Service (Thin Interface)

Role. `service-chatbot` (port 8001) is intentionally **thin**. Every incoming message is forwarded to the NLU service, which owns the conversation brain. The chatbot's only responsibilities are session bookkeeping in Redis, rate limiting (60 messages/hour per user), and call-back hooks for portal field collection.

Why a thin chatbot layer at all? Decoupling presentation from decision logic. Tomorrow when we add a WhatsApp bot or an IVR voice front-end, both can call the same chatbot service and reuse the entire NLU brain without re-implementing

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

state.

Key files:

main.py - FastAPI app, POST /api/v1/chat, POST /api/v1/session/notify, POST /api/v1/session/reset.

session.py - Redis-backed session store (TTL 6 hours, max 20 history turns).

chatbot.py - Field-collection mode using Anthropic Claude (FIELD_EXTRACTION_PROMPT).

config.py - Claude model id, max tokens, temperature, rate limits.

Layer 3 - NLU Service (Conversation Brain)

Role. service-nlu (port 8003) is the heart of the conversation. It does five jobs:

- **Language detection** - Devanagari vs Roman script vs mixed. Decides whether the user is in Hindi, English, or Hinglish.
- **Intent classification** - GREETING, SMALLTALK, COMPLAINT_NEW, COMPLAINT_CONTINUE, STATUS_CHECK, CLARIFICATION_REPLY, ABUSE, OFF_TOPIC.
- **Entity extraction** - phone, pincode, aadhaar, consumer_no, account_no, vehicle_no, email, organisation, person, duration, infrastructure.
- **Slot filling and state machine** - tracks where the conversation is (IDLE - COLLECTING - READY - AWAITING_LOCATION - CONFIRMING - FIELD_COLLECTION - SUBMITTED).
- **Domain keyword extraction** - maps surface words ("bijli", "transformer", "■■■■", "FIR") to domain hints used by the classifier.

AI model used: Claude Haiku 4.5 (model id claude-haiku-4-5-20251001).

Specifically, we use Anthropic's Claude Haiku for two things: (a) the high-level NLU decision (intent + completeness + next state + reply text) at temperature 0.2 and max 600 tokens, and (b) field extraction during stage 6, where Claude reads a free-form citizen reply and returns a structured {"extracted": ..., "reply": ...} JSON object.

Why Claude Haiku specifically:

- **Latency.** Haiku 4.5 returns full structured replies in 600 ms - 1.2 s. This matters because the user is waiting on a chat bubble.
- **Cost.** Haiku is roughly 10x cheaper per token than Sonnet and 60x cheaper than Opus - critical when we are paying per chat turn for a public-facing system.
- **Multilingual quality.** Anthropic's models handle Devanagari and Hinglish code-mixing significantly better than GPT-3.5 class models, which we found drop accents and mistranslate idioms ("3 din se nahi aa rahi" - "hasn't been coming for 3 days" used to get translated as "hasn't come for 3 dinners" by smaller models).
- **Structured-output reliability.** Haiku 4.5 reliably emits clean JSON without code fencing in our prompt.
- **Prompt caching.** The system prompt + decision-tree templates are cached on Anthropic's side, slashing per-turn cost.

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

Alternatives we considered:

- **Mistral 7B/Codestral.** Cheaper but noticeably worse on Hinglish code-mixing in our internal tests; we keep the key in .env but no longer call it.
- **GPT-4o-mini.** Comparable cost, slightly higher latency at the time of writing, weaker on Devanagari proper nouns.
- **Local IndicBERT / MuRIL for everything.** Great for classification but cannot generate conversational replies - they are encoders, not decoders.
- **Hand-rolled intent classifier.** Tried first. Fragile - users say things like "yaar pichli baar wali shikayat ka kya hua?" which is a status-check phrased like small talk.

Domain dictionary excerpt (keywords -> domain hints):

```
DOMAIN_DICT = {
    # Electricity
    "bijli": "electricity", "■■■■": "electricity",
    "transformer": "electricity", "meter": "electricity",
    "power cut": "electricity", "voltage": "electricity",

    # Water
    "paani": "water", "■■■■": "water", "tanker": "water",

    # Roads (word-boundary matched)
    "sadak": "roads", "■■■■": "roads", "pothole": "roads",
    "■■■■": "roads", "highway": "roads",

    # Safety / encroachment / noise
    "shor": "safety", "loudspeaker": "safety", "kabza": "safety",
    ...
}
```

Subtle but important: matching uses regex word boundaries so that "ration" inside the word "duration" does not falsely trigger the PDS domain. This bug bit us early in testing.

State machine (excerpt):

```
class StateEnum(str, Enum):
    IDLE = "IDLE"
    COLLECTING = "COLLECTING"          # gathering complaint text
    READY = "READY"                    # complaint complete
    AWAITING_LOCATION = "AWAITING_LOCATION" # waiting for GPS pin
    CONFIRMING = "CONFIRMING"          # asking user to confirm submission
    FIELD_COLLECTION = "FIELD_COLLECTION" # filling portal-specific fields
    SUBMITTED = "SUBMITTED"
```

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

Layer 4 - Classifier Service (Department + Sub-category + Priority + Sentiment)

Role. service-classifier (port 8004) consumes the normalised text + domain hints from NLU and produces four parallel predictions: **department** (one of 25 classes), **sub_category** (one of 142 classes), **sentiment** (Neutral, Frustrated, Distressed) and **priority** (Low, Med, High, Critical).

Model used: fine-tuned MuRIL (google/muril-base-cased) + four sklearn classification heads.

MuRIL (Multilingual Representations for Indian Languages) is a Google-published BERT-family encoder pre-trained on 17 Indian languages *including transliterations*. That last property is the reason we use MuRIL instead of plain mBERT or XLM-R: Indian users write Hindi in both Devanagari and Roman script ("bijli" vs "■■■■■■") and MuRIL is the only widely-available encoder that places both forms close together in embedding space.

Architecture:

We freeze the MuRIL encoder and use it only to extract sentence embeddings. Embeddings are computed as **mean pooling over non-padding tokens** at max sequence length 128 (we tested 256 and 512; the gain was inside the noise floor but inference latency doubled). The 768-d vector is then fed into four independent sklearn classifiers, one per output head:

```
# Department and sub_category (>30 classes each)
Pipeline([
    ("scaler", StandardScaler()),
    ("sgd", SGDClassifier(
        loss="modified_huber", # calibrated probabilities
        alpha=1e-4,
        max_iter=200,
        n_jobs=-1,
        class_weight="balanced",
    )),
])

# Sentiment and priority (<10 classes)
Pipeline([
    ("scaler", StandardScaler()),
    ("lr", LogisticRegression(
        max_iter=2000, C=1.0,
        solver="saga", n_jobs=-1)),
])
```

Why this two-tier design (encoder + sklearn) and not a fine-tuned classification head on MuRIL?

- Speed of iteration. Re-training a single sklearn head takes seconds on CPU; fine-tuning the whole MuRIL encoder takes hours on GPU.

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

- Re-use of embeddings. We cache the entire training-set embedding matrix in `embeddings_cache.npy` and reuse it across all four heads.
- Calibrated probabilities. We need confidence scores not just argmax, because below 0.40 confidence we ask the user a clarifying question. SGD with modified_huber gives well-calibrated probabilities; vanilla cross-entropy on a small classification head does not.
- Imbalanced classes. `class_weight="balanced"` in sklearn is one line; the equivalent in PyTorch is a sampler dance.

Three-tier inference pipeline:

When a request lands, the classifier tries three sources of signal in priority order:

Tier 1 - layer2_confirmed. If the chatbot already asked the user "is this about Electricity / Water / ...?" and the user picked, the chatbot forwards that `layer2_category`. We override department with the user's pick, confidence 0.92, model tag `layer2_confirmed`.

Tier 2 - MuRIL inference. Embed -> 4 classifiers -> top-3. If MuRIL's top-1 disagrees with the NLU domain hints, we trust the hints (synthetic confidence 0.75, model tag `muril+hint`).

Tier 3 - rule-based fallback. If MuRIL artifacts are missing or `USE_RULE_BASED_CLASSIFIER=true` is set, we use the `DOMAIN_TO_DEPT` static map. Confidence 0.85 if hints exist, 0.50 otherwise.

Department label set (25 classes):

Aadhaar (UIDAI), Agriculture & Farmers Welfare, Banking (DFS), Consumer Affairs, EPFO, Education (Higher / School), Electricity, GST (CBIC), Health & Family Welfare, Housing & Urban Affairs, Income Tax (CBDT), Insurance (DFS), Passport (MEA), Pension & Pensioners' Welfare, Petroleum & LPG, Police, Postal, Public Distribution (PDS), Public Safety & Encroachment, RTO / State Transport, Railways, Roads & Transportation, Telecom, Waste Management, Water Supply

Sub-categories: 142 fine-grained labels including "Bill Discrepancy", "Transformer Fault", "Cyber Fraud", "Aadhaar Linking", "Ration Card Issue", "FIR Registration", "PM-Kisan Payment", "TDS Mismatch", "Ambulance Delay", "Pothole or Road Damage", "Streetlight", "Sewage Overflow", "Public Nuisance", and so on across all 25 departments.

Clarification loop:

If department confidence drops below 0.40 the classifier returns `needs_clarification=True` with a Hindi clarifying question built from the top-3 departments. The orchestrator routes this back to the chatbot, which asks the user. After the user picks, the next classification call has `layer2_category` set and Tier 1 takes over with high confidence. We cap at 2 clarification rounds per complaint to avoid infinite loops.

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

Layer 5 - Routing Service (Portal Selection)

Role. service-routing (port 8005) takes a department tag + district + state and returns the single most appropriate government portal out of 70 registered options.

Why a separate routing service? Portal selection is a policy problem (which authority is responsible for what) not an ML problem - so the right tool is a hand-curated registry with deterministic matching logic, not a model. Keeping it separate also lets policy admins update portals.csv without touching code.

The portal registry (portals.csv):

70 portals across three tiers:

- **30 Central portals** (covers ALL_INDIA) - e.g. CPGRAMS (P001), PMO (P002), Income Tax e-Nivaran (P003), RailMadad (P007), UIDAI Aadhaar (P011), CGHS, EPFO grievance, NHAI Sukhad Yatra, PFRDA, India Post.
- **30 State (MP) portals** (covers ALL_MP) - e.g. CM Helpline 181 (P031), MP Lokayukta (P035), MP Jal Nigam (P040), MP School Education (P042), MP Agriculture (P049), MP RTO, MP Police state, etc.
- **10 Regional portals** (district-specific) - MPPKVVCL Bhopal Discom (P061), MPPaschim Indore Discom (P062), MPMKVVCL Jabalpur (P063), Bhopal Municipal Corp (P064), Indore Municipal Corp (P065), Gwalior Nagar Nigam, Jabalpur Nagar Nigam, Ujjain Nagar Nigam, Sagar Nagar Nigam, Bhopal Police (P070), Indore Police (P071).

CSV columns: portal_id, portal_name, portal_level, authority_name, covers_districts (pipe-separated), website, has_online, classifier_dept_tags (pipe-separated), complaint_categories, required_fields (pipe-separated), helpline, whatsapp, fallback_portal_id.

Decision algorithm:

Hierarchical: **regional > state-specific > state catch-all > central-specific > central catch-all**. More specific portals are preferred because local authorities resolve faster and citizens expect local handling. We define a specificity key (fewer dept tags handled = more specialised) and break ties by that.

```
def find_portal(dept_tag, district, state):
    candidates = [p for p in portals if dept_tag in p.classifier_dept_tags and p.has_online]

    if state in {"madhya pradesh", "mp", "m.p."}:
        # Tier 1: Regional with district match
        regional = [p for p in candidates
                    if p.portal_level == "Regional" and district in p.covers_districts]
        if regional: return min(regional, key=_specificity_key)

        # Tier 2: State-specific (ALL_MP) excluding catch-alls
        state_p = [p for p in candidates
                  if p.portal_level == "State" and p.portal_id not in _CATCH_ALL_IDS]
        if state_p: return min(state_p, key=_specificity_key)

        # Tier 3: MP state catch-all (P031)
        # Tier 4: Central-specific
```

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

```
# Tier 5: National catch-all (P001 CPGRAMS)
...

# Non-MP user: skip MP tiers
central = [p for p in candidates
            if p.portal_level == "Central" and p.portal_id not in _CATCH_ALL_IDS]
return min(central, key=_specificity_key) if central else _portal_by_id("P001")
```

Worked example - power cut in Bhopal:

dept_tag=ELECTRICITY, district=Bhopal, state=Madhya Pradesh. Candidates that handle ELECTRICITY: P061 (MPPKVVCL Bhopal, Regional), P062 (Indore Discom, Regional), P063 (Jabalpur Discom, Regional), P031 (CM Helpline, State catch-all), P001 (CPGRAMS, Central catch-all). Tier 1 match: P061 covers Bhopal. Result: **P061 MPPKVVCL Bhopal Discom**. The user filed a power cut complaint - it goes to the actual electricity board for Bhopal, not to a national catch-all where it would be re-forwarded down the chain over weeks.

Layer 6 - Semantic Deduplication (S-BERT)

Role. Before we file a complaint, we ask: "has this exact complaint already been filed?" - either by this user (block re-submission) or by other users in the same district (aggregate so the dashboard can show frequency). This stage lives in the gateway service, between routing and field-collection.

Model used: paraphrase-multilingual-MiniLM-L12-v2 from sentence-transformers. 117 MB, 384-dimensional vectors, supports 50+ languages including Hindi and Hinglish in the same vector space.

Why S-BERT and not bag-of-words or exact-string match?

- "mera bijli connection nahi aa raha" and "power is not coming at my house" should be detected as the same complaint. Lexical matching cannot do this; semantic embeddings can.
- Multilingual sentence-transformers were trained on parallel corpora explicitly so that translation pairs land near each other in embedding space.
- 384 dims is small enough to store in a Postgres FLOAT8[] column without ballooning row size.
- Inference is cheap - a few milliseconds per text on CPU.

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

Logic:

```
SIMILARITY_THRESHOLD = 0.85

# Candidate set: same dept + sub_cat + district, last 180 days, unresolved
for cand in candidates:
    sim = cosine_similarity(own_embedding, cand.text_embedding)
    if sim >= SIMILARITY_THRESHOLD:
        if cand.user_id == this_user:
            block_resubmission()          # same user, same complaint
        else:
            cluster_count += 1            # different user, aggregate signal
```

Why threshold 0.85? Empirically calibrated on our small test set. Below 0.80 we get false positives ("no electricity" and "no water" being too similar in multilingual space). Above 0.92 we miss obvious rephrasings. 0.85 is the sweet spot - we will re-tune on real production data.

Layer 7 - Portal Field Collection

Role. Each portal expects different fields. CPGRAMS wants *complainantName, address, pinCode, mobileNumber, emailId, aadhaarNo, ministryId, grievanceText*. MPPKVCL wants *IVRS consumer number, full address of connection, complaint type*. We refuse to dump that complexity on the citizen. Instead, after routing has selected the portal, the chatbot enters FIELD_COLLECTION state and asks for each missing field conversationally.

Pre-fill from conversation. Many fields are already inferable from the conversation so far: name and mobile from the user profile, address from the location pin, description from the original complaint, complaint type from the classifier. Only genuinely portal-specific fields (consumer number, IVRS, PNR) are asked. This dropped average questions-per-complaint from ~7 to ~2.

Field extraction prompt (verbatim from chatbot/prompts.py):

```
You are collecting form fields for a government grievance portal.
The user is answering one field at a time.

Current field to collect: {field_name}
User language: {language}
Next field (if any): {next_field}

Rules:
- Extract the value for "{field_name}" from the user message.
- If the user says they don't know / "skip" / non-answer -> extracted=null.
- Keep reply to 1 short sentence in the user language.
- If extracted is not null and next_field given, mention next field in reply.
- If extracted is null, give a brief hint about where to find this field.

Respond ONLY with JSON (no markdown):
{"extracted": "<value or null>", "reply": "<1 sentence>"}
```

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

11 validators live in chatbot/validators.py - regex checks for pincode (6 digits), mobile (10 digits, starts 6-9), aadhaar (12 digits, Verhoeff checksum), email (RFC-lite), PAN, IFSC, GST, vehicle number, consumer number, account number, PNR. If a validator rejects, we re-ask once with a corrective hint.

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

Layer 8 - Submission Service (Adapter Pattern)

Role. service-submission (port 8006) is the integration layer that actually pushes complaints to the chosen portal's API. Different portals expose different schemas (and sometimes no API at all - just a form). We solved this with a classic **adapter pattern**: one base class, one adapter per portal family.

4-method adapter contract (PortalAdapter base class):

```
class PortalAdapter(ABC):
    portal_id: str
    portal_name: str

    @abstractmethod
    async def submit(self, portal_fields, uco_meta) -> SubmissionResult:
        """Build payload -> POST -> return ticket."""

    @abstractmethod
    async def fetch_status(self, ticket_id) -> StatusResult:
        """Poll the portal API for current status."""

    def transform_fields(self, portal_fields, uco_meta) -> dict:
        """Map our generic field labels -> portal-specific keys."""
        return {**portal_fields, **uco_meta}

    def map_canonical_status(self, raw_status: str) -> str:
        """Portal-specific status strings -> UCO canonical status."""
        return _STATUS_MAP.get(raw_status.lower(), "IN_PROGRESS")
```

Registered adapters:

- **CPGRAMSAdapter** - handles P001 (CPGRAMS) and P002 (PMO Public Grievance). Maps to CPGRAMS field names like complainantName, grievanceText, ministryId.
- **MPCM181Adapter** - handles P031 (MP CM Helpline 181). Maps to district / tehsil / village fields.
- **MPPKVCLAdapter** - handles P062 / P065 (MP electricity discoms). Needs IVRS consumer number.
- **GenericMockAdapter** - fallback for any portal not yet wired up; generates a realistic ticket id with the portal-type prefix ("ELE/", "POL/") for demo purposes.

Retry policy: 7-attempt exponential backoff.

```
_RETRY_DELAYS = [60, 120, 300, 900, 3600, 14400, 86400]
#                1m   2m   5m   15m   1h   4h   24h

try:
    await submission_service.submit(payload)
except Exception:
    if attempt < len(_RETRY_DELAYS):
        loop.call_later(_RETRY_DELAYS[attempt],
            lambda: trigger_submission(complaint_id, payload, attempt + 1))
    else:
        mark_complaint_failed_and_alert_admin()
```

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

Why this exact backoff curve? Most portal failures are transient (gateway timeout, captcha, brief outage). 1 and 2 minute retries handle those. The 15-minute and 1-hour gaps cover scheduled portal maintenance windows. 4-hour and 24-hour final attempts cover all-day outages. After 8 attempts we mark the complaint failed and surface it to the admin dashboard for human review.

Layer 9 - Status Tracker

Role. service-tracker (port 8007) polls each submitted complaint's portal at adaptive intervals until it reaches a terminal state, normalises portal-specific status strings into UCO canonical statuses, and notifies the citizen.

Adaptive polling schedule:

```
def poll_interval_seconds(submitted_at):
    age = now() - submitted_at
    if age < timedelta(days=1): return 3600      # every 1 hour
    if age < timedelta(days=7): return 21600     # every 6 hours
    return 86400                                # daily after that
```

Fresh complaints get checked frequently because that's when status changes happen. Old complaints get checked daily so we don't hammer portals for stale records.

Status normaliser - portal statuses are messy strings in mixed languages:

```
_STATUS_MAP = {
    "submitted": "PENDING", "registered": "PENDING", "received": "PENDING",
    "under review": "IN_PROGRESS", "in progress": "IN_PROGRESS",
    "forwarded to ministry": "IN_PROGRESS", "officer assigned": "IN_PROGRESS",
    "awaiting response": "AWAITING_USER",
    "■■■■■■■■ ■■■■■■■■■■": "AWAITING_USER",          # Hindi
    "resolved": "RESOLVED", "closed": "RESOLVED", "disposed": "RESOLVED",
    "rejected": "REJECTED", "closed no action": "REJECTED",
}
```

Citizen notifications (mock Twilio WhatsApp): on every status change we send a bilingual message - Hindi first, English fallback. On RESOLVED status the message asks "Reply 1 (haan) or 2 (nahi)" - this enters the feedback loop in the gateway, which records satisfaction and (on "2") escalates back to the admin queue.

Cross-cutting Layer - SHA-256 Tamper-Evident Audit Chain

Every meaningful event in the lifecycle of a complaint - chat_message, submit_attempt, submit_failed, submit_exhausted, feedback_requested, status_changed - is written to complaint_events with a SHA-256 hash that includes the previous event's hash.

```
raw = f"{prev_hash or ''}{event_type}{json.dumps(details, sort_keys=True)}{now_iso}"
event_hash = hashlib.sha256(raw.encode()).hexdigest()
```

This means an auditor (or the citizen themselves) can recompute every hash from the genesis event to verify that no event was deleted, re-ordered, or tampered with. Each chain is per-complaint, kept in the relational

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

complaint_events table. The right column on the admin dashboard renders this chain visually.

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

4. Department / Government Dashboard Flow - Layer by Layer

The second product surface in Margdarshan.ai is the dashboard used by reviewing departments and oversight bodies. It is a separate routing inside the same React SPA, accessible after an admin user logs in (the `is_admin` flag on the user record gates this).

Layer 1 - Admin Authentication

Same login screen as citizens but the post-login router checks `users.is_admin`. The default admin account (`admin@margdarshan.ai` / `Admin@1234`) is bootstrapped on first gateway start if it doesn't exist. Auth uses bcrypt password hashing and PyJWT-signed tokens stored in `localStorage`.

Layer 2 - Stats Overview

Hero cards at the top show: total complaints, duplicate-complaint count, breakdown by status, breakdown by department, breakdown by district, and the 10 most recent complaints with summary, status, department, and timestamp. Endpoint: `GET /api/v1/admin/stats`. Backed by Postgres aggregation queries with appropriate indices (`idx_complaints_dedup`, `idx_complaints_created_at`).

Layer 3 - Portal Registry View

Full list of the 70 portals with `portal_id`, name, level (Regional/State/Central), authority, districts covered, website link, online filing flag, classifier dept tags, complaint categories, helpline, and the live count of complaints currently routed to each portal. Filterable by department, district, level. Endpoint: `GET /api/v1/admin/portals`.

Why this matters: The most common operational question for a state oversight body is "are we sending complaints to the right places?" This view answers that in one screen, and the per-portal complaint count flags portals that may be misconfigured.

Layer 4 - Complaints List with Filters

Filterable grid of all complaints in the system. Filters: status, department, district. Columns: `complaint_id`, summary, status badge, department, sub_category, district, `portal_id`, `ticket_id`, `user_name`, `user_mobile`, priority badge, sentiment badge, `is_duplicate` flag, `duplicate_count`, `created_at`. Pagination at 50 rows by default. Endpoint: `GET /api/v1/admin/complaints?status=&department;=&district;=&limit;=&offset;=`.

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

Layer 5 - Complaint Detail Drill-Down

Clicking any complaint opens a full detail view with five sections:

- **Filer info** - name, mobile, email, district. Anonymisable for cross-department views (PII masked except for last 4 digits of mobile).
- **Classification** - department, sub_category, priority, sentiment, confidence, plus the top-3 alternatives the classifier considered.
- **Portal routing explanation** - which portal_id was chosen, why (matching tier, district match, fallback path).
- **Portal fields** - the key-value pairs we collected and posted to the portal.
- **Audit chain** - the entire SHA-256 hash-chained event log for this complaint, rendered as a timeline.

Endpoint: GET /api/v1/admin/complaints/{id}.

Layer 6 - Review Form (Thumbs Up / Down + Corrections)

On every complaint detail page, reviewing officers can rate the AI's decisions. The form accepts:

- rating (positive / negative) - the headline feedback
- classification_correct (bool)
- correct_department (text override)
- correct_sub_category (text override)
- correct_priority (Low/Med/High/Critical override)
- sentiment_correct (bool)
- reviewer_notes (free text)

Posted to POST /api/v1/admin/complaints/{id}/review. The data lands in the complaint_reviews table, which forms our training feedback loop: every misclassification flagged by a real reviewer becomes a candidate row in the next MuRIL fine-tuning dataset.

Layer 7 - Duplicate Filer Detection

Surfaces (a) repeat complaints filed by the same person, and (b) clusters of complaints across different users in the same district that are semantically similar (cosine ≥ 0.85). The dashboard shows: complaint_id, filer name, mobile, filed_at, status, similarity score, plus the portal fields from each filing - which makes it easy to spot whether multiple citizens are reporting the same incident (e.g., a transformer fault that 30 households are calling about).

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

Layer 8 - Routing Explanation

For every complaint, we can answer "why was this sent to Portal X?". The explanation includes: (i) department tag from classifier, (ii) district/state from location pin, (iii) the matching tier that fired (Regional, State-specific, State catch-all, Central-specific, National catch-all), (iv) which specific portal-rule matched. This is a transparency feature that makes the system auditable for citizens and oversight bodies alike.

Layer 9 - API Trace / Demo Page

An animated 5-step page that shows the live API exchange for a single complaint: Classification request/response, Routing decision JSON, Portal submission request/response, Current tracking status. Built for demos but also a useful debug surface during incidents. Endpoint: GET /api/v1/demo/trace/{complaint_id}.

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

5. AI / ML / NLP Foundations Used in the System

This section explains the conceptual machinery in plain terms - useful for evaluators who want to know what each technical term means and why it is the right pick.

5.1 Transformers and Encoder Models

A **transformer** is a neural network architecture (Vaswani et al., 2017) that processes an input sequence by computing **self-attention** between every pair of tokens. The output for any token is a weighted blend of every other token's representation, with the weights learned during pretraining. This is why transformers handle context far better than the RNNs they replaced.

An **encoder-only** transformer (BERT, MuRIL, XLM-R) takes a sequence in and produces a contextual vector for every token. A **decoder-only** transformer (GPT-class models, Claude, Llama) takes a sequence in and produces the next token, one at a time. We use encoder-only for classification (we need representations, not text generation) and decoder-only for the conversational chatbot (we need fluent text generation in three languages).

5.2 MuRIL (encoder we use for classification)

MuRIL is a BERT-base architecture (12 layers, 768 hidden size, 12 attention heads, ~236M params) pretrained by Google Research on 17 Indian languages. Crucially, it was pretrained on both native scripts *and* the Roman-script transliterations that Indian users actually type. Its tokeniser is wordpiece, vocab size 197k. The published BPC perplexities show it outperforms mBERT and XLM-R on 9 of 10 Indic NLP benchmarks.

We use **mean pooling** (averaging the final-layer hidden states across all non-padding tokens) as our sentence representation rather than the [CLS] token, because empirical results in the sentence-BERT literature show mean pooling is marginally better when the encoder is frozen and you feed the embeddings into a downstream classifier.

5.3 Sentence-BERT (S-BERT) for Deduplication

S-BERT (Reimers and Gurevych, 2019) is a fine-tuning recipe that produces sentence embeddings where **semantic distance corresponds to cosine distance**. Vanilla BERT embeddings are not directly comparable that way. The specific model we use, paraphrase-multilingual-MiniLM-L12-v2, is a 12-layer student distilled from a multilingual XLM-R teacher, trained on the parallel paraphrase corpus across 50+ languages. 384 dims, 117 MB, sub-10 ms inference per sentence on CPU.

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

5.4 Claude Haiku 4.5 (decoder model for dialogue)

Claude is Anthropic's family of large language models. **Haiku 4.5** (model id claude-haiku-4-5-20251001) is the smallest tier - optimised for low latency and low cost. It is a decoder-only transformer with extended-context support (200k tokens), trained with Constitutional AI techniques. We invoke it via Anthropic's REST API with temperature 0.2 (low randomness - we want consistent intent classification, not creative text), max 600 tokens per response.

Two prompt patterns in our system: (1) **structured NLU prompt** in service-nlu - returns a typed JSON object with intent, language, complaint_buffer, completeness, next_state, reply_to_user; (2) **field extraction prompt** in service-chatbot - returns {extracted, reply} for a single portal field per turn.

5.5 Cosine Similarity (the metric we threshold on)

Cosine similarity between two vectors a and b is $(a \cdot b) / (|a| * |b|)$. It is in $[-1, 1]$ for general vectors and in $[0, 1]$ for non-negative embeddings. It measures the angle between the vectors, ignoring magnitude - which is exactly what you want when comparing semantic content irrespective of sentence length.

5.6 Calibrated Probabilities and Why We Care

An argmax of softmax scores tells you the most likely class, but it does not tell you *how sure* the model is. We need the latter so we can route low-confidence cases through a clarification loop instead of acting on a wrong guess. `SGDClassifier(loss="modified_huber")` gives calibrated probabilities cheaply; we threshold at 0.40 on the department head.

5.7 Pub/Sub Pattern (event-driven orchestration)

Each service emits `PipelineEvent` messages with (complaint_id, stage, status, payload) to a single Redis channel. The gateway orchestrator subscribes and triggers the next stage. This decouples services in time and space - we can add, remove, or restart any service without rewiring the others.

5.8 Adapter Pattern (object-oriented portal integration)

A classical Gang-of-Four pattern. The 4-method `PortalAdapter` interface lets new portals slot in by writing one subclass; the rest of the submission service knows nothing about portal-specific schemas.

5.9 Hash Chains (tamper evidence)

Inspired by blockchain merkle structures but linear: every event includes the SHA-256 of the previous event in its own hash input. Mutating any historical event invalidates every subsequent hash, which a verifier can detect in linear time.

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

6. Model Training, Evaluation, and Accuracy

6.1 Classifier Training Dataset

We curated **v3_enriched.csv**, a 3,564-row labelled dataset of Indian grievances written in Hindi, English, and Hinglish. Each row has: complaint_text, department (one of 25), sub_category (one of 142), priority, sentiment. The dataset was bootstrapped from CPGRAMS publicly disclosed grievance excerpts, augmented with synthetic Hinglish paraphrases generated by Claude Sonnet and reviewed by a human (myself) before inclusion.

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

6.2 Training Pipeline (train_muril.py)

Hyperparameters:

Parameter	Value	Why
Encoder	google/muril-base-cased (frozen)	Best multilingual coverage of Indian languages incl. transliteration.
Max sequence length	128 tokens	Median complaint length 41 tokens; 128 covers p95 with no truncation loss.
Pooling	Mean over non-padding tokens	Empirically better than CLS for frozen-encoder classification.
Batch size	32	Fits in 8 GB RAM during embedding extraction.
Device	MPS (Apple Silicon) or CPU	GPU not required - MuRIL is small enough.
Department head	SGDClassifier(modified_header)	Calibrated probabilities, balanced class weights.
Sub-category head	SGDClassifier(modified_header)	Same - 142 classes need probabilities for top-k.
Sentiment / priority head	LogisticRegression(saga)	Few classes, smooth solver handles imbalance.
Train / val split	85 / 15 stratified by department	Preserve class balance in val set.
Eval metric	weighted F1	Robust to class imbalance, matches the misclassification cost we care about.

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

6.3 Reported Accuracy

Head	Classes	Weighted F1 (val)	Notes
Department	25	0.91	Strong signal from domain hints + dedicated keyword overrides.
Sub-category	142	~0.85	Range 0.60 to 1.00 per class - rare classes dominate the spread.
Sentiment	3	0.78	Frustrated and Distressed often overlap lexically.
Priority	4	0.83	Critical override by keyword set is rule-based and 100% recall.

6.4 Why F1 and not accuracy?

Class distribution is heavily imbalanced (Electricity and Water Supply dominate; Aadhaar-Linking has 50x fewer rows than Power Outage). Accuracy would let the model coast by always predicting majority classes. Weighted F1 penalises the model for ignoring minority classes proportionally to their support.

6.5 Confusion Matrix Observations

- Most frequent confusion: Water Supply -> Roads & Transportation when complaints mention *nali* (drain) - both domains legitimately match. We bias toward Water in the rule-based fallback when *paani* or *tanker* also appears.
- Noise / loudspeaker complaints used to land in Waste Management because of an early keyword overlap. Fixed by moving *shor* / *loudspeaker* / *dj* into a dedicated *safety* domain and routing it to Public Safety & Encroachment.
- Aadhaar Linking vs Aadhaar Seeding - finer-grained sub-category confusion. We accept this; the routed portal is the same.

6.6 Production Status of MuRIL

MuRIL inference is wired up and benchmarked but currently **disabled** in deployment via `USE_RULE_BASED_CLASSIFIER=true` in `.env`. The rule-based path (domain hints + keyword maps + layer-2 confirmation) is achieving department-level accuracy comparable to MuRIL on our hackathon test traffic, with simpler ops and faster cold-start. Re-enabling MuRIL is a one-line env-var toggle and a service-classifier rebuild.

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

6.7 How We Continuously Improve

Every classification with a thumbs-down review in the admin dashboard becomes a labelled training row in the next training cycle. The `complaint_reviews` table is structured exactly to be joined back to `complaints.pipeline_data` and emitted as a fresh CSV row. This closes the loop between operations and model quality.

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

7. Software Engineering Concepts in Play

7.1 Microservices vs Monolith

Each pipeline stage is its own service, deployed independently. Inter-service contracts are typed FastAPI bodies validated by Pydantic; the shared Pydantic models live in a single shared-schema package. Why not a monolith: separate runtime dependencies (PyTorch in classifier, anthropic SDK in chatbot, just httpx in location), independent scaling, independent failure domains.

7.2 FastAPI Everywhere

Every Python service is FastAPI 0.115 on uvicorn 0.30. Why FastAPI: (a) Pydantic validation gives us schema-correct request and response bodies for free, (b) async support out of the box - which matters because every service is I/O bound on Postgres, Redis, or external APIs, (c) auto-generated OpenAPI docs at /docs on every service.

7.3 Async I/O

All inter-service calls use httpx.AsyncClient. Database access uses asyncpg. The orchestrator loop is a single async task that handles the entire event stream. This lets a single uvicorn worker handle dozens of in-flight complaints concurrently without thread overhead.

7.4 Authentication and Authorization

Password storage: bcrypt with cost factor 12 (~150 ms to hash, safe against rainbow tables). Tokens: PyJWT-signed with HS256 and a 32-character secret from JWT_SECRET env var. Admin gating: the user record has an is_admin boolean - we never trust the client to set that.

7.5 Database Schema Highlights

Six tables: users, complaints (with JSONB pipeline_data + FLOAT8[] text_embedding), messages, complaint_events (the audit chain), complaint_reviews (admin feedback). A partial index idx_complaints_dedup (department, sub_category, district, status, created_at DESC) WHERE status NOT IN ('resolved','rejected') makes the dedup query $O(\log n)$ over relevant rows only.

7.6 Frontend Patterns

- **Context + custom hooks** for global state (useAuth, useChat, usePipeline) - the right tool for a small but live-updating SPA.
- **Component-driven UI** - 20+ small composable components rather than a few mega-components.

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

- **WebSocket for live updates** - usePipeline opens `ws://gateway/ws/pipeline/{id}` and pushes events into the right panel as they arrive.
- **react-i18next** with three resource files - the entire UI swaps language with one dropdown click.
- **Leaflet + OSM** for the location pin - free, no API key, works offline once tiles are cached.

7.7 DevOps

- Docker Compose for the whole stack - `docker compose up -d` brings up 10 services + Postgres + Redis in under 30 seconds.
- Healthchecks on Postgres and Redis so dependent services wait for readiness, not just port-open.
- Volume mounts for hot-reloading frontend code and re-using the host's HuggingFace cache (no re-download of MuRIL on every rebuild).
- End-to-end smoke test script at `scripts/smoke_test.sh` exercises auth, chat, geocode, and admin endpoints.

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

8. Comparative Analysis with Existing Systems

Several public grievance systems exist in India today. We benchmarked Margdarshan.ai against the four most prominent.

Capability	CPGRAMS	MP CM Helpline 181	MyGov	Per-utility portals (MP PKVVCL etc.)	Margdarshan.ai
Conversational input	No - long form	Yes (IVR)	No - structured forms	No - form per portal	Yes - chat + voice
Multilingual (Hindi / Hinglish)	Limited - Hindi UI only	Hindi IVR	11 Indian languages but English forms	Mostly English	Hindi + English + Hinglish, end-to-end
Auto portal selection	No - user picks ministry	Centralised - one portal	No	No - one portal each	Yes - 70 portals, regional > state > central
Semantic deduplication	No	No	No	No	Yes - S-BERT cosine 0.85
Field pre-fill from conversation	No	Operator collects	No	No	Yes - location/name/desc pre-filled
Citizen status notifications	Email	SMS	Email	Varies	Bilingual WhatsApp + chat
AI classification visibility / audit	Closed	Closed	Closed	Closed	Open - top-3 alternatives + reviewer override

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

Capability	CPGRAMS	MP CM Helpline 181	MyGov	Per-utility portals (MP PKVVCL etc.)	Margdarshan.ai
Tamper-evident audit	No	No	No	No	Yes - SHA-256 chain per complaint
Single citizen surface across portals	No	No (MP only)	No	No	Yes - one chat covers all 70

How we do better - concretely:

- **One door, many portals.** CPGRAMS forwards rejected complaints down the chain over weeks. We pick the right door first time.
- **Citizen language, not bureaucratic language.** A user writes "bijli 3 din se nahi aa rahi" and the system handles it end-to-end. CPGRAMS would force them to translate that into English bullet points.
- **Conversational field collection.** No portal in India today asks you for required fields in chat form - all of them dump you into an HTML form. We collect fields one at a time with hints in your language.
- **Visible AI decisions.** Every classification ships with its top-3 alternatives and confidence. Departments can override and that override becomes training data. No existing system exposes this surface.
- **Hash-chained audit.** Closed systems make trust very hard. We make every event verifiable.
- **Semantic deduplication.** Across the same chamber, citizens often re-file the same complaint in different words. No public portal detects this; we do.

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

9. Future Enhancements and Roadmap

9.1 Production Hardening

- **Real portal integrations.** Replace mock adapters with HTTP submissions to actual portal APIs once we secure department MoUs. CPGRAMS already exposes such an API behind credentialed access.
- **OAuth integrations with DigiLocker** for verified identity proof during sensitive submissions (Aadhaar, passport).
- **Twilio WhatsApp Business** for real outbound messaging (current implementation is logged-only).
- **Rate limiting and abuse detection** beyond the per-hour limit - per-IP, per-user-mobile, per-text-fingerprint to deter mass campaign abuse.

9.2 ML Quality

- **Re-enable MuRIL** with the most recent training set incorporating department reviewer feedback (closing the feedback loop).
- **Active learning** - sample low-confidence classifications for human labelling rather than random.
- **Cross-lingual zero-shot** for languages we have no training data in - Marathi, Bengali, Tamil - using XLM-R as encoder.
- **Reranker for portal routing** - learn from "complaints where the routed portal escalated back to a different one" to improve the routing heuristics.
- **Better dedup** using HNSW (Hierarchical Navigable Small World) over pgvector instead of brute-force cosine - scales to millions of complaints.

9.3 Product Surface

- **Native mobile app** (React Native) - the chat surface translates trivially.
- **WhatsApp bot front-end** - reuse the same chatbot/NLU services; users who don't install apps can still file complaints.
- **Public dashboard** - anonymised, aggregate view of district-level grievance trends. Useful for civic journalism and citizen accountability.
- **Voice IVR** - call a toll-free number, the same NLU brain takes over. Critical for users with low literacy.
- **Document attachment** - photos of potholes, copies of bills - currently the schema supports it but the UI doesn't expose upload yet.
- **Multi-user complaint clustering on the citizen side** - tell a user "23 of your neighbours also reported this; we're escalating as a group".

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

9.4 Department Side

- **SLA timers** per portal - flag complaints crossing deadlines automatically.
- **Officer assignment workflow** - department reviewer can route specific complaints to specific officers from within the dashboard.
- **Trend dashboards** with department-level KPIs (resolution rate, time-to-first-response, citizen satisfaction).
- **API for departments** - departments can pull their queued complaints into their existing case-management software.

9.5 Trust and Privacy

- **End-to-end encryption** for sensitive complaints (sexual harassment, whistleblowing).
- **Differential privacy** on public-dashboard aggregates so individuals cannot be re-identified.
- **Right-to-erasure tooling** - users can purge their account; the audit chain handles tombstones rather than deletes.
- **External hash anchoring** - periodically write the latest chain head to a public blockchain so even our database admin cannot rewrite history undetected.

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

10. Appendix - APIs, Schemas, Deployment

10.1 Gateway API Surface (public endpoints)

Method	Path	Purpose
POST	/api/v1/auth/register	Create user, return JWT.
POST	/api/v1/auth/login	Authenticate (email or mobile), return JWT.
GET	/api/v1/auth/me	Return current user profile.
POST	/api/v1/chat	Send a chat message, get reply + pipeline state.
POST	/api/v1/complaint/attach-location	Pin location to a complaint.
POST	/api/v1/session/reset	Clear chatbot + NLU session.
POST	/api/v1/session/restore/{cid}	Restore prior session for editing.
GET	/api/v1/complaints	List user's complaints.
GET	/api/v1/complaints/{id}	Full complaint detail incl. pipeline_data.
GET	/api/v1/complaints/{id}/messages	Chat history for a complaint.
WS	/ws/pipeline/{id}	Real-time pipeline event stream.
GET	/api/v1/admin/stats	Admin: aggregated counts and recent complaints.
GET	/api/v1/admin/portals	Admin: 70-portal registry.
GET	/api/v1/admin/complaints	Admin: filtered complaint list.
GET	/api/v1/admin/complaints/{id}	Admin: full complaint detail + audit chain.
POST	/api/v1/admin/complaints/{id}/review	Admin: submit AI-decision review (thumbs up/down + corrections).
POST	/api/v1/admin/complaints/{id}/status	Admin: override complaint status.
GET	/api/v1/demo/trace/{id}	Demo: full API exchange JSON for a complaint.

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

Method	Path	Purpose
POST	/api/v1/location/reverse-geocode	Proxy to service-location.
GET	/api/v1/location/pincode/{pin}	Proxy to service-location.

10.2 Unified Complaint Object (UCO) - core schema

```
class UCO(BaseModel):
    complaint_id: str
    user_id: str
    session_id: str
    text_raw: str
    text_normalized: Optional[str]
    text_for_classifier: Optional[str]
    language: Optional[LanguageEnum] # en / hi / hinglish
    location: Optional[LocationData] # lat/lon/pincode/ward/district/state
    entities: Entities # phone/aadhaar/email/...
    keywords: list[str]
    domain_hints: list[str]
    classification: Optional[Classification] # dept, sub_cat, priority, sentiment, confidence
    portal: Optional[Portal] # portal_id, name, jurisdiction, required_fields
    status: Status # PENDING/IN_PROGRESS/RESOLVED/...
    created_at: datetime
    submitted_at: Optional[datetime]
    resolved_at: Optional[datetime]
```

10.3 Pipeline Event Schema

```
class PipelineEvent(BaseModel):
    complaint_id: str
    stage: StageEnum # stage_0_chat / stage_2_nlu / stage_3_classify / ... / stage_10_status
    status: EventStatusEnum # started / completed / failed / skipped
    payload: dict[str, Any]
    error: Optional[str]
    created_at: datetime
```

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

10.4 Database Tables

users	id, name, gender, email, mobile, phone, password_hash, address, sub_locality, locality, state, district, pincode, country, is_admin, created_at
complaints	complaint_id (UUID PK), user_id, summary, intent, state, department, sub_category, district, portal_id, ticket_id, status, text_embedding (FLOAT8[384]), pipeline_data (JSONB), created_at, updated_at INDEX (department, sub_category, district, status, created_at DESC) WHERE status NOT IN ('resolved','rejected')
messages	id, complaint_id, user_id, session_id, role, content, seq (BIGSERIAL), created_at
complaint_events	event_id, complaint_id, event_type, actor, details (JSONB), prev_hash, event_hash, created_at
complaint_reviews	id, complaint_id, classification_correct, correct_department, correct_sub_category, correct_priority, sentiment_correct, reviewer_notes, rating (positive/negative), created_at

10.5 Deployment

From repo root with Docker Desktop running:

```
$ docker compose up -d

# Then open the citizen UI:
$ open http://localhost:5173

# Admin login:
#   email:    admin@margdarshan.ai
#   password: Admin@1234

# End-to-end smoke test:
$ bash scripts/smoke_test.sh
```

Margdarshan.ai

Multilingual AI Grievance Redressal Platform

Comprehensive Technical Documentation

BGI Hackathon 2026 - Team: Merge Conflict - Problem BT1P1

10.6 Environment Variables

```
ANTHROPIC_API_KEY=sk-ant-api03-... # Claude Haiku
LOCATIONIQ_API_KEY=... # optional, fallback to OSM
POSTGRES_USER=grievance
POSTGRES_PASSWORD=...
POSTGRES_DB=grievance
REDIS_HOST=redis REDIS_PORT=6379
JWT_SECRET=... # 32+ chars
ADMIN_EMAIL=admin@margdarshan.ai # bootstrapped on first start
ADMIN_PASSWORD=Admin@1234
USE_RULE_BASED_CLASSIFIER=true # false -> MuRIL
CHATBOT_URL=http://service-chatbot:8001
NLU_URL=http://service-nlu:8003
CLASSIFIER_URL=http://service-classifier:8004
ROUTING_URL=http://service-routing:8005
SUBMISSION_URL=http://service-submission:8006
TRACKER_URL=http://service-tracker:8007
LOCATION_URL=http://service-location:8002
VITE_GATEWAY_URL=http://localhost:8000
```

Document end. Generated programmatically from the live codebase on 2026-05-12.